19. HTTP from the CLI: curl and friends

19. HTTP from the CLI: curl and friends

Phase: Phase 6 — Networking
Estimated time: 1 day
Slug: `19-curl-http`

Learning objectives

- Fetch URLs with curl
- Inspect headers and status codes
- POST JSON for APIs
- Follow redirects consciously

Prerequisites

Complete prior numbered lessons. Prefer a real Linux VM/container for Linux-specific topics. Type every command yourself — muscle memory beats skimming.

Why this matters

HTTP is the protocol behind web apps, REST APIs, health checks, package mirrors, and cloud metadata services. A browser hides the details; `curl` shows them. With `curl` you can see status codes, headers, redirects, TLS problems, and timing, and you can script health checks that decide whether a deployment worked.

Knowing the status-code families lets you read an incident at a glance: 2xx worked, 3xx redirected, 4xx the client was wrong, 5xx the server broke.

Core concepts

Flag	Meaning
`-s`	Silent: hide progress meter
`-S`	Show errors even with `-s`
`-I`	HEAD request, headers only
`-i`	Include response headers in output
`-L`	Follow redirects
`-o file`	Write body to file
`-w '%{http_code}'`	Print a field after finishing
`-f`	Fail (exit 22) on HTTP 4xx/5xx
`-X POST -d`	Method and body
`-H 'K: V'`	Add header
`--max-time 5`	Total timeout in seconds
`-k`	Skip TLS verification (avoid in production)

Status code families:

Range	Meaning	Examples
1xx	Informational	101
2xx	Success	200 OK, 201, 204
3xx	Redirect	301, 302, 304
4xx	Client error	400, 401, 403, 404, 429
5xx	Server error	500, 502, 503, 504

One request, step by step:

```
curl -i https://example.com/
   |
   v   DNS -> TCP connect -> TLS handshake
GET / HTTP/1.1
Host: example.com
   |
   v
HTTP/1.1 200 OK            <-- status line (the code http_ok tests)
Content-Type: text/html    <-- headers
<blank line>
<html>...                  <-- body
```

Worked examples

Headers only:

```
curl -sI https://example.com
```

Sample output:

```
HTTP/2 200
content-type: text/html; charset=UTF-8
cache-control: max-age=604800
```

Extract just the status code, the pattern used in health checks:

```
curl -s -o /dev/null -w '%{http_code}\n' https://example.com
curl -s -o /dev/null -w '%{http_code}\n' https://example.com/missing
```

Sample output:

```
200
404
```

Follow a redirect and show the final URL:

```
curl -sL -o /dev/null -w '%{http_code} %{url_effective}\n' http://example.com
```

Sample output:

```
200 https://example.com/
```

Send JSON to an API and fail on errors:

```
curl -fsS -X POST https://httpbin.org/post \
  -H 'Content-Type: application/json' \
  -d '{"name":"ada"}' | head -4
```

Sample output:

```
{
  "args": {},
  "data": "{\"name\":\"ada\"}",
```

A health-check function in the spirit of the exercise:

```
http_ok() { [ "$1" = "200" ] && echo yes || echo no; }
code=$(curl -s -o /dev/null -w '%{http_code}' https://example.com)
http_ok "$code"
```

Sample output:

```
yes
```

Timing breakdown:

```
curl -s -o /dev/null -w 'dns=%{time_namelookup} connect=%{time_connect}
total=%{time_total}\n' https://example.com
```

Sample output:

```
dns=0.012 connect=0.034 total=0.190
```

Mini-lab

1. Run `curl -sI https://example.com` and name the status line, a header, and the blank line that ends headers.
2. Fetch a 404 with `-w '%{http_code}'` and note the difference from the exit status of curl (still 0).
3. Repeat with `-f` and print `$?`; it should be 22.
4. Use `curl -sIL http://github.com` and count the redirect hops.
5. Save a page: `curl -s -o page.html https://example.com` and check `wc -c page.html`.
6. POST data to `https://httpbin.org/post` and read the echoed JSON.
7. Add `--max-time 2` against an unroutable address such as `10.255.255.1` and observe exit 28 (timeout).
8. Write a loop that checks three URLs and prints the code for each.

Decision guide / failure modes

Symptom	Cause	Next step
`Could not resolve host` (6)	DNS problem	`dig`, check resolver
`Connection refused` (7)	Nothing listening	`ss -tlpn` on server
Timeout (28)	Firewall or slow server	`--max-time`, trace path
301/302 with empty body	Redirect not followed	Add `-L`
401/403	Auth or permission	Check token and headers
502/503/504	Proxy cannot reach backend	Check upstream service
TLS error (60)	Bad or expired certificate	`openssl s_client`, fix cert

Curl exits 0 for any HTTP response unless you pass `-f`. For automation use `-fsS` plus `--max-time`, and test `%{http_code}` when you need to distinguish 200 from other 2xx codes. Never use `-k` as a permanent fix.

Common mistakes

- curl without timeouts hanging automation
- Leaking tokens in shell history
- Ignoring TLS verification permanently with -k

Exercises

Work in `exercises/19-curl-http/`. Read that folder's `README.md` fully before coding.

```
cd exercises/19-curl-http
chmod +x test.sh exercise.sh
./test.sh
```

What you will build

Implement http_ok in exercise.sh. It receives an HTTP status code as its first argument. When the code is exactly 200 it must print yes, and for anything else (404, 500, 301, an empty string, or no argument) it must print no. Compare it as a string with quotes around the variable so an empty argument does not cause a syntax error. The function does not need to run curl; the test supplies the codes. After ./test.sh passes, combine it by hand with curl -s -o /dev/null -w '%{http_code}' against a real URL.

Study loop

- **A (Drill):** make `./test.sh` pass
- **B (Apply):** run 2–3 lab commands from this lesson on a real Linux host; paste notable output into your notes
- **C (Challenge):** rewrite the exercise from memory the next day

Do not open `solutions/19-curl-http/` until you have an honest failing attempt.

Self-check

Try to answer without scrolling up:

1. What does -I do?
2. Why set max-time?
3. How send JSON POST?

Further reading

- https://curl.se/docs/manual.html